Python Database Practicals

SQLite Practical Questions 14–25

Inventory database operations and a menu-driven To-Do List Manager.

How to use: Run Practicals 14–22 in order for the inventory database. Run Practical 23 before Practical 24 or 25. Practical 25 is a complete menu-driven program and can be run on its own.

Practical 14: Connect to inventory.db and Create a Cursor

Question: Connect to inventory.db using sqlite3 and create a cursor for executing SQL commands.

```
import sqlite3

# Connect to the database (creates it if it does not exist)
conn = sqlite3.connect("inventory.db")

# Create cursor
cur = conn.cursor()

print("Connected to inventory database!")
print("Cursor created successfully.")

conn.close()
```

Sample output:

```
Connected to inventory database!
Cursor created successfully.
```

Practical 15: Create the Products Table

Question: Create products table with id, name, quantity and price; commit changes and close the connection.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    price REAL NOT NULL
)
""")

conn.commit()
print("Products table created successfully!")

conn.close()
```

Sample output:

```
Products table created successfully.
```

Practical 16: Insert Single and Multiple Products

Question: Insert single and multiple products using execute() and executemany(), and display affected rows and last inserted ID.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

# Insert one product
cur.execute(
    "INSERT INTO products(name, quantity, price) VALUES (?, ?, ?)",
    ("Pen", 100, 10.5)
)
print("Single insert - affected rows:", cur.rowcount)
print("Last inserted ID:", cur.lastrowid)

# Insert multiple products
product_list = [
    ("Notebook", 80, 45.0),
    ("Pencil", 150, 5.0),
    ("School Bag", 30, 550.0)
]

cur.executemany(
    "INSERT INTO products(name, quantity, price) VALUES (?, ?, ?)",
    product_list
)
print("Multiple insert - affected rows:", cur.rowcount)

conn.commit()
conn.close()
```

Sample output:

```
Single insert - affected rows: 1
Last inserted ID: 1 (example)
Multiple insert - affected rows: 3
```

Practical 17: Retrieve Products Using SELECT

Question: Retrieve all products, find a product by name and display products with quantity below 60.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

# Display all products
print("All products:")
cur.execute("SELECT * FROM products")
for row in cur.fetchall():
    print(row)

# Find a product by name
print("\nSearch product:")
cur.execute("SELECT * FROM products WHERE name = ?", ("Pen",))
print(cur.fetchone())

# Products with quantity below 60
print("\nProducts with quantity below 60:")
cur.execute("SELECT * FROM products WHERE quantity < ?", (60,))
for row in cur.fetchall():
    print(row)

conn.close()
```

Sample output:

```
All products:
(1, 'Pen', 100, 10.5)
...
Search product:
(1, 'Pen', 100, 10.5)

Products with quantity below 60:
(3, 'Pencil', 150, 5.0)  # Example data may differ
```

Practical 18: Update Product Quantity and Price

Question: Update the quantity and price of products and display affected rows.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

cur.execute("""
UPDATE products
SET quantity = ?, price = ?
WHERE id = ?
""", (120, 12.0, 1))

print("Affected rows:", cur.rowcount)

conn.commit()
conn.close()
```

Sample output:

```
Affected rows: 1
```

Practical 19: Delete a Product by ID

Question: Delete a product from the products table by its ID and display affected rows.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

cur.execute("DELETE FROM products WHERE id = ?", (2,))

print("Deleted rows:", cur.rowcount)

conn.commit()
conn.close()
```

Sample output:

```
Deleted rows: 1
```

Practical 20: Demonstrate commit() and rollback()

Question: Demonstrate commit and rollback by inserting products and verifying the changes.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

# Insert and commit: this record is saved
cur.execute(
    "INSERT INTO products(name, quantity, price) VALUES (?, ?, ?)",
    ("Marker", 40, 25.0)
)
conn.commit()
print("Marker inserted and committed.")

# Insert and rollback: this record is cancelled
cur.execute(
    "INSERT INTO products(name, quantity, price) VALUES (?, ?, ?)",
    ("Temporary Item", 10, 99.0)
)
conn.rollback()
print("Temporary Item insert rolled back.")

# Verify records
cur.execute("SELECT name FROM products WHERE name IN (?, ?)",
            ("Marker", "Temporary Item"))
print("Saved records:", cur.fetchall())

conn.close()
```

Sample output:

```
Marker inserted and committed.
Temporary Item insert rolled back.
Saved records: [('Marker',)]
```

Practical 21: Backup Database Using iterdump()

Question: Back up inventory.db into inventory_backup.sql using iterdump().

```
import sqlite3

conn = sqlite3.connect("inventory.db")

with open("inventory_backup.sql", "w", encoding="utf-8") as file:
    for line in conn.iterdump():
        file.write(line + "\n")

print("Database backup created: inventory_backup.sql")

conn.close()
```

Sample output:

```
Database backup created: inventory_backup.sql
```

Practical 22: Export Products to CSV

Question: Export all products to products_export.csv with column headers.

```
import sqlite3
import csv

conn = sqlite3.connect("inventory.db")
cur = conn.cursor()

cur.execute("SELECT id, name, quantity, price FROM products")
rows = cur.fetchall()

with open("products_export.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(["id", "name", "quantity", "price"])
    writer.writerows(rows)

print("Products exported to products_export.csv")

conn.close()
```

Sample output:

```
Products exported to products_export.csv
```

Practical 23: Create To-Do Tasks Table

Question: Connect to todo_list.db, create the tasks table with required fields, commit changes and close the connection.

```
import sqlite3

conn = sqlite3.connect("todo_list.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    completed INTEGER NOT NULL DEFAULT 0
)
""")

conn.commit()
print("Tasks table created successfully!")

conn.close()
```

Sample output:

```
Tasks table created successfully.
```

Practical 24: Create Functions to Manage Tasks

Question: Create Python functions to add, view, complete and delete tasks with error handling and proper connection management.

```
import sqlite3

DB_NAME = "todo_list.db"

def add_task(title, description=""):
    try:
        with sqlite3.connect(DB_NAME) as conn:
            conn.execute(
                "INSERT INTO tasks(title, description) VALUES (?, ?)",
                (title, description)
            )
        print("Task added successfully.")
    except sqlite3.Error as e:
        print("Database error:", e)

def view_tasks():
    try:
        with sqlite3.connect(DB_NAME) as conn:
            rows = conn.execute(
                "SELECT id, title, description, completed FROM tasks"
            ).fetchall()

        if not rows:
            print("No tasks found.")
        for task in rows:
            status = "Completed" if task[3] else "Pending"
            print(task[0], task[1], "-", status, "|", task[2])
    except sqlite3.Error as e:
        print("Database error:", e)

def complete_task(task_id):
    try:
        with sqlite3.connect(DB_NAME) as conn:
            cur = conn.execute(
                "UPDATE tasks SET completed = 1 WHERE id = ?",
                (task_id,)
            )
        print("Task completed." if cur.rowcount else "Task ID not found.")
    except sqlite3.Error as e:
        print("Database error:", e)

def delete_task(task_id):
    try:
        with sqlite3.connect(DB_NAME) as conn:
            cur = conn.execute(
                "DELETE FROM tasks WHERE id = ?", (task_id,)
            )
        print("Task deleted." if cur.rowcount else "Task ID not found.")
    except sqlite3.Error as e:
```

```
        print("Database error:", e)

# Example function calls
add_task("Study Python", "Practice SQLite programs")
view_tasks()
complete_task(1)
delete_task(1)
```

Sample output:

```
Task added successfully.
1 Study Python - Pending | Practice SQLite programs
Task completed.
Task deleted.
```

Practical 25: Menu-Driven To-Do List Manager

Question: Create a menu-driven main program loop for the To-Do List Manager.

```
import sqlite3

DB_NAME = "todo_list.db"

def setup_database():
    with sqlite3.connect(DB_NAME) as conn:
        conn.execute("""
        CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            description TEXT,
            completed INTEGER NOT NULL DEFAULT 0
        )
        """)

def add_task():
    title = input("Enter task title: ").strip()
    description = input("Enter description: ").strip()

    if not title:
        print("Title cannot be empty.")
        return

    with sqlite3.connect(DB_NAME) as conn:
        conn.execute(
            "INSERT INTO tasks(title, description) VALUES (?, ?)",
            (title, description)
        )
    print("Task added.")

def view_tasks():
    with sqlite3.connect(DB_NAME) as conn:
        rows = conn.execute(
            "SELECT id, title, description, completed FROM tasks"
        ).fetchall()

    if not rows:
        print("No tasks available.")
    for task in rows:
        status = "Completed" if task[3] else "Pending"
        print(task[0], task[1], "|", status, "|", task[2])

def complete_task():
    try:
        task_id = int(input("Enter task ID to complete: "))
        with sqlite3.connect(DB_NAME) as conn:
            cur = conn.execute(
                "UPDATE tasks SET completed = 1 WHERE id = ?",
                (task_id,)
            )
        print("Task completed." if cur.rowcount else "Task not found.")
    except ValueError:
        print("Please enter a valid number.")

def delete_task():
    try:
        task_id = int(input("Enter task ID to delete: "))
        with sqlite3.connect(DB_NAME) as conn:
            cur = conn.execute(
                "DELETE FROM tasks WHERE id = ?", (task_id,)
            )
        print("Task deleted." if cur.rowcount else "Task not found.")
    except ValueError:
        print("Please enter a valid number.")

def main():
    setup_database()
```

```
    while True:
        print("\n--- TO-DO LIST MENU ---")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Complete Task")
        print("4. Delete Task")
        print("5. Exit")

        choice = input("Enter your choice: ")

        try:
            if choice == "1":
                add_task()
            elif choice == "2":
                view_tasks()
            elif choice == "3":
                complete_task()
            elif choice == "4":
                delete_task()
            elif choice == "5":
                print("Thank you!")
                break
            else:
                print("Invalid choice. Try again.")
        except sqlite3.Error as e:
            print("Database error:", e)

if __name__ == "__main__":
    main()
```

Sample output:

```
--- TO-DO LIST MENU ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
Enter your choice: 5
Thank you!
```